

ECE358: Computer Networks

Lab 2 - HTTP and Sockets

1. Introduction

In this lab you will be implementing a subset of the Hyper Text Transfer Protocol (HTTP). HTTP is the main protocol used by applications such as web browsers to transfer data to and from web servers. HTTP is an application layer protocol that relies on parsing bytes sent between a client and a server. We are going to focus on HTTP/1.1, which is foundational to all subsequent versions of HTTP even if it is a bit deprecated. HTTP/1.1 is particularly simple to understand because the messages, passed as bytes, are parsed directly into human-readable text before the server and client handle the HTTP data. It is also used in about a third of all websites in the world, so it's a good starting point.

Lab Overview and Learning Outcomes

In this lab, you will:

1. Write a server in Python that implements a meaningful subset of HTTP/1.1
2. Test your server by writing a client that sends and receives requests
3. Explore the main HTTP verbs GET, HEAD, POST, PUT, and DELETE to send plain text data between the client and the server
4. Understand the HTTP message format, including the request, headers, and body

2. Our Simplified HTTP

HTTP is a rich protocol that can transfer many types of data of arbitrary length. In this lab, we are simplifying the protocol quite a bit, mainly because once you get a simplified version running it is just more of the same work to get the whole protocol running. Here are the simplifications we are implementing:

1. We are using plain HTTP, not HTTPS. This is not a course on encryption!
2. All messages will be less than or equal to 1024 bytes long, including all header and body information. You are allowed to hard code this maximum length in your Python code, and the starter code is already set up to send and receive messages up to this length.
3. We are transferring only plain text in utf-8 format.
4. Paths to all resources will be given starting with '/', for example, '/file.txt', or '/files/file1.txt'. This '/' is the root *of the server's filesystem*, not of the computer's file system. So for example if your server is in `/home/mstachow/server.py`, then a request for `/files/file1.txt` should look in `/home/mstachow/files/`, not the root folder `/files`.

5. All HTTP requests require a **Host** to be specified. You must properly format your HTTP requests to have a **Host** header field, but you may ignore that field in your server. This means that the server must read and parse out the **HOST**, but you do not need to implement multiple hosts. For simplicity, you may use **www.example.com** as your **Host** in all requests.

You will be implementing the following methods (also known as “verbs” in HTTP parlance) and status codes.

Methods

Method Name	Purpose	Success Codes	Error Codes
GET	Retrieve a resource.	200 OK	404 Not Found
HEAD	Like GET but only returns headers (no body).	200 OK	404 Not Found
POST	Submit data for processing (e.g., form data).	201 Created	400 Bad Request
PUT	Create or replace a resource at the given URI.	200 OK (indicates the resource existed already and has been updated), 201 Created (indicates the resource did not exist and now has been created)	400 Bad Request
DELETE	Remove a resource at the URI.	200 OK	404 Not Found, 401 Unauthorized, 403 Forbidden

Status Codes

Code	Name	Meaning
200	OK	Request succeeded; response body contains the result.

Code	Name	Meaning
201	Created	New resource created successfully.
400	Bad Request	Request could not be understood (malformed syntax).
401	Unauthorized	Client authentication is missing.
403	Forbidden	Client not allowed to perform this action.
404	Not Found	Resource does not exist at the given URL.
405	Method Not Allowed	The requested method was not allowed by the server.

You may notice that we are not implementing any of the 5xx status codes. These codes indicate internal server errors, which are extremely hard to duplicate in a lab setting since you are writing both the client and the server. Since you are writing both, to trigger an internal server error you would need to intentionally write code that breaks your server. Instead, the only "server" error we are implementing is 405 - method not allowed. To test this, you would simply send a nonsense method name outside of the list of methods we are working with.

3. In-Depth Specifications for Each Method

Understanding how to Format an HTTP Message: Client-side

HTTP messages must be properly formatted to be understood by the client and server programs. Messages are made up of lines, separated by the exact string sequence `\r\n`. In general, a message looks like this:

```

VERB <resource_path> HTTP/1.1 \r\n
Host: <hostname> \r\n
<optional headers, one per line, terminated by \r\n>
\r\n
<optional body, depending on the VERB>

```

For example, here is an HTTP PUT request for a hypothetical server sent by the client:

```

PUT /api/index/test_file.txt HTTP/1.1 \r\n
Host: www.example.com \r\n
Content-Type: text/plain \r\n
Content-Length: 3 \r\n
\r\n
abc

```

HTTP distinguishes between the header information and the body via a blank line, which is why the extra `\r\n` must be present after all header fields have been specified. In what follows, we will describe the specifications that your client and server must meet for this lab. We recommend working on this in the order in which we specify the verbs, but it is up to you.

Understanding how to Format an HTTP Response: Server-side

The server's response is similar to the client's message, but the verb is omitted since the server is not making a request. Instead, it begins with a status line indicating whether the request succeeded, and potentially providing more information. Here is the general format:

```
HTTP/1.1 <status code> <status name>\r\n
Content-Type: text/plain \r\n
Content-Length: <length in bytes of the body, omit if there is no body> \r\n
<optional other headers if needed> \r\n
\r\n
<optional body if the server's response requires it>
```

For example, here is a simple HTTP 404 Not Found error with a descriptive message in the body:

```
HTTP/1.1 404 Not Found\r\n
Content-Type: text/plain \r\n
Content-Length: 50 \r\n
\r\n
The requested resource was not found on the server
```

In the specifications below, we are giving you the requirements *for this lab*. That is, this is not a one-to-one copying of the entire HTTP protocol itself. Therefore, if we say that something is not allowed or the order must be respected, you should read this as saying “in the context of this lab, you must implement the specification”, not “HTTP in general has these restrictions”.

The GET request

HTTP's GET request is a request sent to the server to obtain the raw bytes of a resource. This is what you use when you request a simple HTML page from a web server, but it can be more general than this.

Client

- The resource path must be specified as above
- The Host header must be present
- No other headers are allowed
- No body is present

Server The server looks for the requested resource at the request path. If it is not found, it returns a 404 Not Found error with “404 Not Found” in the body. Otherwise, it returns a 200 OK status code. The content length is the number of bytes in the body, and the body contains the contents of the file.

The HEAD request

HTTP's HEAD request is used to query a server to determine, among other things, if the resource exists and its length.

Client The HEAD request's format is identical to that of the GET request

Server The server's response is identical to that of the GET response, but Content-Length and the body is omitted. In the case of a 404 Not Found response, you may omit the body as well.

The POST request

HTTP's POST request is used to send raw data to a server for later processing. It is extremely common in web forms, where things like a username or other user data are entered and sent to the server. Usually, the web server extracts relevant information from the request and routes it to backend software for actual processing. For example, if we POST our name on a web form, we might expect that it eventually gets saved in a database. In this lab we are simply going to echo the data back to the client to ensure that it was properly sent.

Client

- The resource path must be `simple_form`
- The Host header must be present
- The Content-Type must be `text/plain`
- The Content-Length is the length of the body being sent
- The body is plain text according to the format below.

In this lab, we are going to implement a simple colon-separated POST request. You must send a body that contains a string, a colon, and another string. Your strings must contain alphanumeric (and potentially spaces) data only - no punctuation, and especially no colons. The first string is interpreted as a username, and the second string is interpreted as a description. Here is an example:

```
mike:lab instructor
```

Server The server's response depends on whether the request was properly formed or not.

Assuming that the request is properly formed, the server responds as follows:

- Status code 201 Created
- Content-Type is 'text/plain'
- Content-Length is the length of the body
- The body is the string: `Name is: {name} and Description is {description}` where name and description are parsed from the request

There are two possible ways for the request to be malformed in this lab: either the `resource_path` is not `simple_form`, or the body of the request does not fit the format described above. During testing, “does not fit the format” means that a colon is missing. Otherwise the request will still conform to the standards in this lab. We won’t, for example, send multiple lines or strings longer than the hard-coded maximum. Either way, the server should respond with a status code of 400 Bad Request, with the body being the string “400 Bad Request”. The Content-Type of this response would still be `text/plain` and the Content-Length is the length of the body.

The PUT request

HTTP’s PUT request is used to store files on the server. For this reason, real servers need to ensure that the user initiating this request is in fact authorized to do so. We are going to ignore this complication for the PUT request!

Client Other than changing the request type to PUT, the client’s PUT request is format identical to the POST request. In this lab, we are going to allow PUT requests only to the folder `/user/`, so the `resource_path` must begin with `/user/`. For example, to write the body of the request to the file “myfile.txt”, the `resource_path` would be specified as `/user/myfile.txt`. You may assume that the client always puts files into `/user/`, not any subfolders, and subfolder behaviour will not be tested.

Server The server’s behaviour depends on what is in the client’s request.

The server responds with a 400 Bad Request message identically to a bad POST request if any of the following are met:

- The client attempts to PUT a file into a folder other than `/user/`
- The client does not provide a filename

If the client’s request is valid, there are two possibilities: either the client is attempting to create a new file or overwrite an existing one.

When the client attempts to create a new file, the server should respond with a 201 Created status response, with the body containing the exact string `201 Created`.

When the client attempts to PUT data into a file that already exists, the old file must be overwritten entirely, and the server responds with a 200 OK status response, with the body containing the exact string “200 OK”

The DELETE request

DELETE allows a user to delete a resource from the server. Clearly, this is not something that a real server should just allow from any user - the user needs the appropriate permissions, authentication and so on - otherwise anyone could

delete anything! In this lab, we are going to implement a very basic (and *very* insecure) authentication scheme for this request. The server should have in its base directory a file named `users.txt`. This file contains username,password pairs, separated by commas, one per line. It would look like this:

```
alice,super_secret
bob,plaintext_passwords_are_great
```

Client The client sends a username-password pair as a header using an Authorization: Basic header. Because it is a bit weird to figure out, here is a Python function that correctly formats a DELETE request. You may feel free to use it in your client code:

```
import base64
def DELETE(resource_path, resource_host, username=None, password=None):
    auth_line = ""
    if username is not None and password is not None:
        token = base64.b64encode(f"{username}:{password}".encode()).decode()
        auth_line = f"Authorization: Basic {token}\r\n"
    return (
        f"DELETE {resource_path} HTTP/1.1\r\n"
        f"Host: {resource_host}\r\n"
        f"{auth_line}\r\n"
    ).encode()
```

The Authorization token is sent (in plaintext, which is why we typically prefer HTTPS for this sort of thing!) and the server must decode it. The format is not arbitrary. The header line must be exactly:

```
Authorization: Basic <base64 encoded token comprised of username:password>\r\n
```

This is part of the HTTP/1.1 standard. The server then has to decode this base64 string and extract the username and password from it.

Server The server has the following behaviour:

1. If the username and password are found in the server's `users.txt` file and the user is trying to delete a file in the `/user/` directory *and* that file exists, the server deletes the file and returns a 200 OK status response as above.
2. If the username and password are found in the server's `users.txt` file but the user is trying to delete a file not in `/user/`, the server returns a 403 Forbidden status response with `403 Forbidden` in the body.
3. If the username and password are found in the server's `users.txt` file and the user is trying to delete a file in the `/user/` directory *but* that file does not exist, the server returns a 404 Not Found status response as above.
4. If the username is not found in the server's `users.txt` file, or the password is incorrect, the server returns a 401 Unauthorized status response with `401 Unauthorized` in the body

5. Any other possibilities will not be tested. If you'd like, you can just return a 403 Forbidden status response for anything else.

4. What you will submit

Rather than writing a report, you are going to submit your server code. Submit a single python file named `<your_uw_username>_server.py` that opens a socket on localhost (127.0.0.1) and binds it to port 8000. For example, I would submit `mstachow_server.py`. You should use the starter code provided to you as the basis for your server code so that you can be sure that this setup step is correct.

Your code will be moved into a new folder that you do not have control over, so you should keep the `SERVER_ROOT = os.getcwd()` line in your code so you know your server's root directory. We will set up the `users.txt` file and the `/user/` folder for you.

Your code will be tested with a client program that you do not have access to, but it will be similar in form to the starter client code that was provided. Your code will be evaluated on correctness. This means that your server must respond as above. The client code will produce requests exactly as shown above. The order of the headers will be exactly as stated above. For your server, however, the exact order of the headers in the response is not important. For example, if our specifications mention Content-Type and Content-Length in that order, but your server responds with Content-Length first, that is acceptable. Additional headers other than what is asked for will be considered incorrect.

Your server will be tested with a single method whose name is not valid. However, the request will be a proper HTTP header + body. That is, line endings will be respected and the general format will be respected. You may simply notice that the method name is not valid and short circuit to a 405 Method Not Allowed response. You do not need to provide a body for this response.

Each request will receive +1 for each correct line of response from the server. For HTTP methods described above in which multiple possible response types are possible, all response types are expected. Each HTTP method's grade will be normalized to a number between 0 and 1, and the average will be taken to compute your overall grade, which will be computed out of 5. This means that some requests will be easier than others to get full marks on!